

MACHINE LEARNING ENGINEERING

FASE 5 | AULA 04 -

ORQUESTRAÇÃO DE CONTÊINERES COM KUBERNETES

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	5
MERCADO, CASES E TENDÊNCIAS	12
O QUE VOCÊ VIU NESTA AULA?	13
REFERÊNCIAS.....	14

EMSE

O QUE VEM POR AÍ?

Nesta aula, você vai dar o salto da execução isolada de containers para a operação distribuída em escala com Kubernetes. Vamos explorar por que a orquestração se tornou necessária para times de ML que precisam manter serviços de inferência disponíveis, atualizáveis e resilientes diante de falhas.

Ao longo do conteúdo, você entenderá como o Kubernetes organiza controle e execução no cluster e como objetos como Pods, Deployments e Services formam a base da operação moderna de aplicações de Machine Learning em produção.

HANDS ON

No hands on, vamos entender sobre o fluxo completo em cluster Kubernetes: início com Pod simples, evolução para Deployment com réplicas e exposição por Service. O objetivo é mostrar na prática como o Kubernetes mantém estado desejado e responde automaticamente a mudanças e falhas.

Vamos também entender como o comportamento do sistema muda quando saímos do modelo “container único” para uma operação orquestrada. Esse ponto é essencial para interpretar incidentes em ambientes reais.

Ao final, conectaremos os principais objetos da plataforma com decisões de engenharia de ML, preparando você para as próximas aulas, onde o foco será implantação de modelos, configuração avançada e escala com mais controle.

SAIBA MAIS

Orquestração como pilar de escala em Machine Learning

Quando um time passa de poucos serviços para dezenas de APIs de inferência, operar containers manualmente deixa de ser sustentável. A complexidade cresce em atualização, descoberta de serviço, recuperação de falha e alocação de recursos. O Kubernetes nasce para resolver exatamente esse tipo de problema com um modelo declarativo e contínuo.

A principal mudança de mentalidade é sair do comando pontual para o estado desejado. Em vez de “subir um container”, você declara como o sistema deve se comportar e permite que a plataforma mantenha esse comportamento mesmo sob falhas.

Arquitetura do cluster

A arquitetura de Kubernetes separa controle e execução; o Control Plane coordena decisões e Worker Nodes que executam workloads. Essa divisão permite escalar e também diagnosticar com precisão.

Quando um Pod não sobe, o problema pode estar no agendamento, na imagem, em limites de recurso ou em política de segurança. Entender a camada correta acelera resolução e reduz tentativas improdutivas.

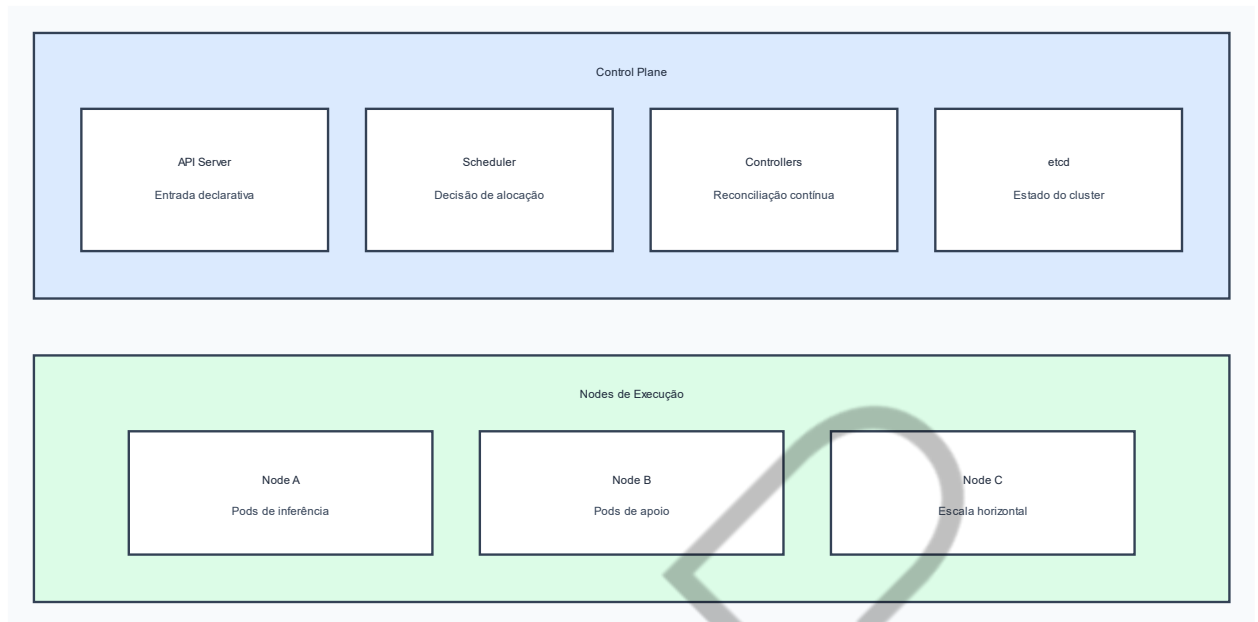


Figura 1 – Arquitetura do Kubernetes
Fonte: Google Imagens (2026)

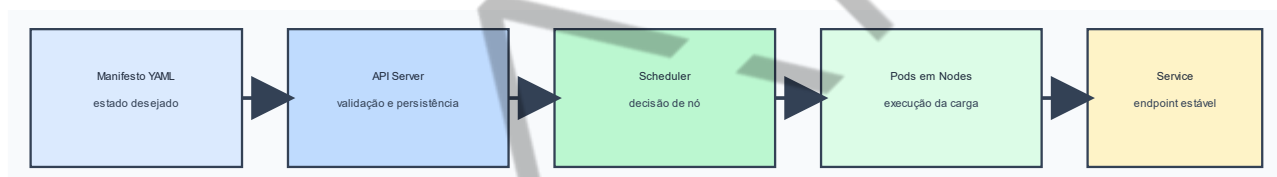


Figura 2 – Fluxo de serviços do Kubernetes
Fonte: Google Imagens (2026)

Reconciliação contínua: confiabilidade por design

O Kubernetes monitora o estado real e compara com o estado desejado. Se houver divergência, controladores atuam automaticamente para convergir novamente. Esse loop de reconciliação torna o ambiente resiliente a eventos comuns como reinício de processo, queda de nó ou indisponibilidade temporária.

Mini-case 1 - falha de nó durante pico: um serviço de recomendação sofre queda de um nó em horário de alta demanda. Como o Deployment está bem definido e há capacidade disponível, o cluster recria Pods em outros nós e mantém o endpoint ativo. O incidente existe, mas o impacto ao usuário é muito menor.

Pod como unidade mínima e efemeridade operacional

Pod é a unidade básica de execução no Kubernetes. Ele é efêmero: pode ser recriado com frequência e não deve carregar estado crítico local. Isso exige que aplicações de ML sejam desenhadas com inicialização idempotente e externalização de estado.

Esse é um ponto central para quem vem de ambientes menos dinâmicos: estabilidade não significa “mesmo processo para sempre”, sim “mesmo serviço disponível apesar de processos mudarem”.

```
apiVersion: v1
kind: Pod
metadata:
  name: ml-inferencia-pod
  labels:
    app: ml-inferencia
spec:
  containers:
    - name: api
      image: ml-inferencia:1.0.0
      ports:
        - containerPort: 8000
```

Código-fonte 1 – Exemplo de manifesto POD para visualizar a unidade mínima (YAML)
Fonte: Elaborado pelo autor (2026)

Deployments: atualização controlada e rollback rápido

Deployments gerenciam réplicas e estratégia de atualização. Em inferência, isso permite publicar novas versões de API ou modelo com menor risco, observando comportamento durante rollout.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: ml-inferencia-deployment
spec:
  replicas: 3
  selector:
    matchLabels:
      app: ml-inferencia
  strategy:
    type: RollingUpdate
    rollingUpdate:
```

```
    maxUnavailable: 1
    maxSurge: 1
  template:
    metadata:
      labels:
        app: ml-inferencia
    spec:
      containers:
        - name: api
          image: ml-inferencia:1.1.0
          ports:
            - containerPort: 8000
```

Código-fonte 2 – Exemplo de manifesto Deployment com estratégia de rolling update (YAML)
Fonte: Elaborado pelo autor (2026)

Estratégias de Deploy:

- Rolling update > Simples e contínua; Pode instanciar gradualmente; Atualizações frequentes.
- Blue/Green > Rollback rápido; Custo temporário maior; Para mudanças críticas.
- Canary > Validação progressiva; Requer observabilidade madura; Para versões de alto risco.

Service: endpoint estável para workloads dinâmicos

Pods mudam de IP ao serem recriados. O Service abstrai essa dinâmica e fornece endpoint estável para consumidores internos ou externos. Esse desacoplamento é fundamental para evitar que clientes precisem “descobrir Pod por Pod”. Sem Service, a arquitetura fica frágil e difícil de operar em escala.

```
apiVersion: v1
kind: Service
metadata:
  name: ml-inferencia-service
spec:
  selector:
    app: ml-inferencia
  ports:
    - protocol: TCP
      port: 80
      targetPort: 8000
  type: ClusterIP
```


Código-fonte 3 – Exemplo de manifesto Service para exposição interna do Deployment (YAML)
Fonte: Elaborado pelo autor (2026)

Escalar bem exige calibrar requests/limits, probes e comportamento de inicialização. Em serviços de ML, o warm-up de modelo pode tornar novas réplicas indisponíveis por alguns segundos ou minutos, afetando a resposta sob pico.

Por isso, a escala precisa ser tratada com observabilidade e testes de carga, não apenas com alteração de número de Pods.

Segurança e governança no nível de plataforma

O Kubernetes oferece mecanismos para segmentação e controle, mas não substitui processo. Namespaces, RBAC, políticas de admissão e revisão de manifests devem fazer parte de uma governança ativa.

Mini-case 2 - bloqueio preventivo de imagem insegura: uma policy de admissão impede deploy de imagem sem assinatura e com vulnerabilidade crítica. O bloqueio acontece antes da produção, reduzindo risco e retrabalho de incidente.

Observabilidade de cluster como requisito de operação

Métricas de nó, eventos de Pod, status de rollout e logs de aplicação devem ser acompanhados de forma estruturada. Sem isso, o time não distingue rapidamente problema de plataforma e problema de aplicação.

Em ambientes de ML, essa distinção é crucial para decidir entre ação de infraestrutura, ajuste de modelo ou rollback de versão.



Figura 3 – Saúde, escalabilidade e atualizações
Fonte: Google Imagens (2026)

DIMENSÃO	OPERAÇÃO MANUAL DE CONTAINERS	OPERAÇÃO COM KUBERNETES
Recuperação de falha	Reativa/manual	Automática por reconciliação
Escala	Dependente de esforço humano	Declarativa e repetível
Atualização de versão	Mais arriscada	Estratégias controladas
Rastreabilidade	Limitada	Alta com objetos versionados
Padronização	Varia por equipe	Estruturada por plataforma

Tabela 1 – Tabela comparativa: manual vs orquestrado
Fonte: Elaborado pelo autor (2026)

Erros comuns de adoção e mitigação

Erro comum 1: tratar Kubernetes como “Docker em outro lugar”.

Correção: projetar para ambiente distribuído e efêmero.

Erro comum 2: ignorar estratégia de rollback.

Correção: versionar Deployments e definir critérios de reversão.

Erro comum 3: publicar sem requests/limits.

Correção: calibrar recursos com métricas reais.

O Kubernetes fornece base sólida para ML em escala quando usado com disciplina arquitetural. Entender arquitetura, reconciliação, objetos e governança permite operar inferência com mais resiliência, previsibilidade e capacidade de evolução contínua.

MERCADO, CASES E TENDÊNCIAS

O Kubernetes se consolidou como padrão de mercado para operação de aplicações distribuídas e o ecossistema de Machine Learning acompanha esse movimento. A principal razão é a capacidade de padronizar deploy, recuperação de falhas e escalabilidade com menos intervenção manual.

Um caso real bastante citado é o do Spotify, que migrou de sua solução interna de orquestração para Kubernetes e reportou ganhos importantes de eficiência operacional e velocidade de provisionamento de serviços. A referência oficial está [aqui](#).

A tendência atual é avançar de “adoção de Kubernetes” para “plataformas internas sobre Kubernetes”, com templates de deploy, políticas de segurança e observabilidade incorporadas por padrão.

O QUE VOCÊ VIU NESTA AULA?

Você consolidou a visão arquitetural do Kubernetes e entendeu como Pods, Deployments e Services estruturam a execução distribuída de aplicações containerizadas.

Também compreendeu por que Kubernetes é base estratégica para ML em escala.



REFERÊNCIAS

BURNS, B.; BEDA, J.; HIGHTOWER, K. **Kubernetes: Up and Running**. Sebastopol: O'Reilly Media, 2022.

HOCKIN, B. **Kubernetes in Action**. Shelter Island: Manning, 2023.

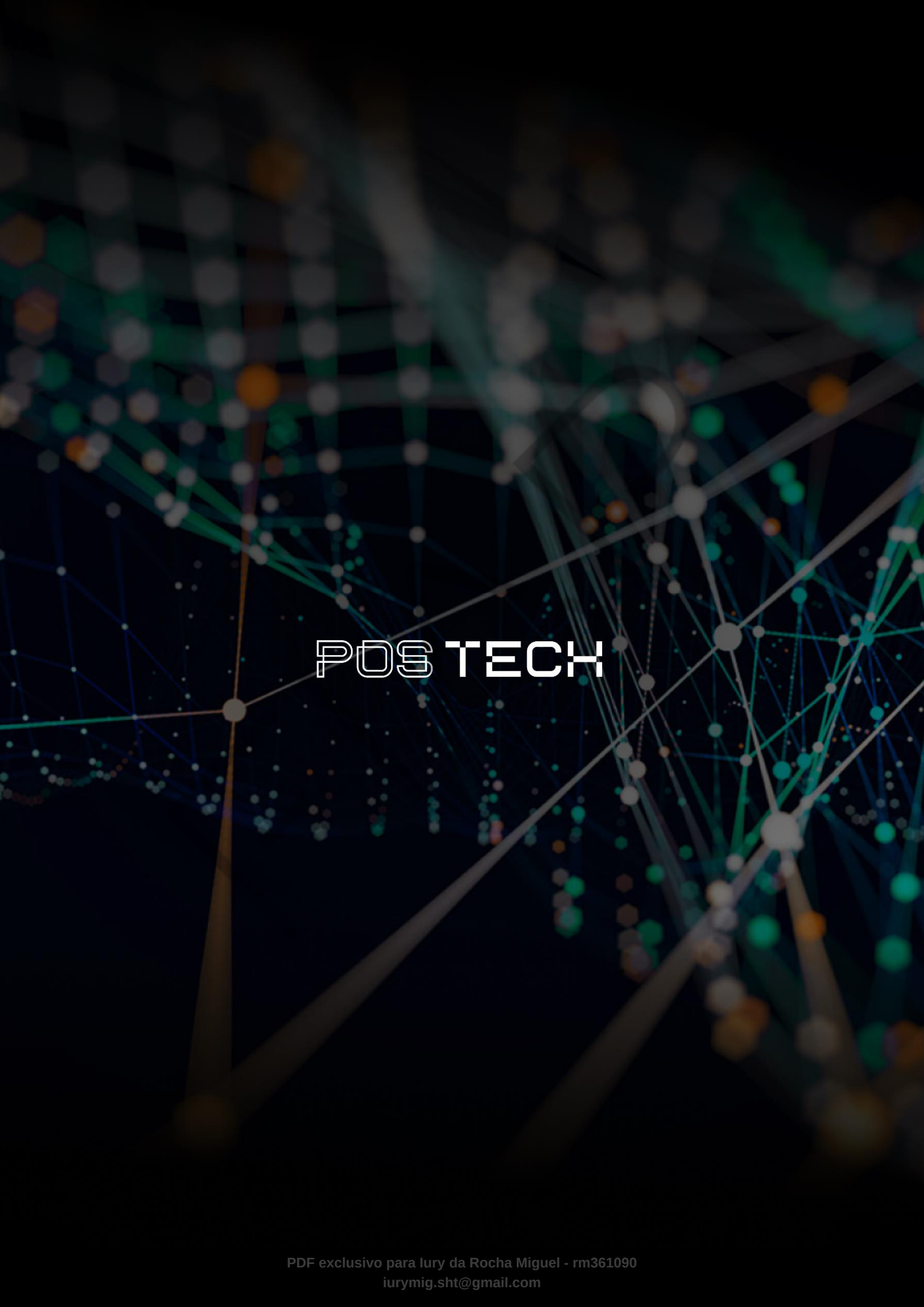
MORGAN, L.; ARUNDEL, J. **Cloud Native DevOps with Kubernetes**. Sebastopol: O'Reilly Media, 2022.

EMANIP

PALAVRAS-CHAVE

DevOps. Containers. Docker. Kubernetes.

EMEND



POSTECH